GET STARTED TODAY

LUKE GINN

AZURE CLOUD MASTERY

Unit 12 Introduction – Ordering Agent Actions

AZURE CLOUD MASTERY

Unit 11 covered MCP tool standardization. Unit 12 covers execution patterns – when agents run one after another vs. at the same time.

- **Review of Unit 11**: You learned to standardize tool connections with MCP. Now you need to decide the order in which agents or tools execute.
- **What This Unit Covers**: You will learn sequential execution (one step completes before next starts) and concurrent execution (multiple steps run simultaneously).
- **Why Execution Order Matters**: Wrong ordering makes agents slow (waiting unnecessarily) or incorrect (running dependent steps out of order).

Exam Takeaway: Sequential vs. concurrent execution is part of Multi-Agent Orchestration domain (35% weighting). The exam tests matching pattern to task dependencies.

Sequential Execution – One Step at a Time

AZURE CLOUD MASTERY

Sequential execution means Agent A completes its entire work, returns a result, and only then Agent B starts with that result as input.

- **Sequential Definition**: Steps run in a chain. Step 2 cannot begin until Step 1 finishes and provides its output. No overlap in time.
- **When to Use Sequential**: Use when Agent B needs the output of Agent A to do its work. Example: Search for a document ID, then fetch the document.
- **Coding Sequential**: Call `result_a = agent_a.process(input)`. Then call `result_b = agent_b.process(result_a)`. Simple linear code.

Exam Takeaway: Sequential execution is for dependent tasks where output of one is input to another. The exam tests identifying dependencies.

Concurrent Execution – Multiple Steps at Once

AZURE CLOUD MASTERY

Concurrent execution means multiple agents or tools run at the same time, and the orchestrator waits for all to complete before combining results.

- **Concurrent Definition**: Steps start together and run in parallel. Total time equals the longest single step, not the sum of all steps.
- **When to Use Concurrent**: Use when steps are independent (no data sharing) and all results are needed. Example: Get weather for three cities simultaneously.
- **Coding Concurrent**: Use `asyncio.gather(agent_a.process_async(), agent_b.process_async())`. Both start at the same time.

Exam Takeaway: Concurrent execution reduces total time for independent tasks. The exam tests when steps can run without waiting for each other.

Dependency – The Key Decision Factor

AZURE CLOUD MASTERY

A dependency exists when one agent or tool needs the output of another to complete its work. Dependencies force sequential execution.

- **Data Dependency Definition**: Agent B needs a value that only Agent A can produce. Example: Agent B needs a customer ID that Agent A retrieves from a database.
- **No Dependency Definition**: Agents work on completely separate data or tasks. Example: Agent A checks inventory. Agent B checks shipping rates. Neither needs the other's output.
- **Identifying Dependencies**: Draw a diagram. If an arrow goes from A to B (A's output flows into B), they must be sequential. If no arrows, they can be concurrent.

Exam Takeaway: Dependencies determine execution pattern. Data flow from one agent to another means sequential. Independent means concurrent.

Sequential Pattern – Search Then Fetch

AZURE CLOUD MASTERY

A common sequential pattern is search first to get an ID, then fetch using that ID. The fetch cannot run without the search result.

- **Step 1 – Search**: Agent calls `search_documents("budget report")`. Search returns a list of document IDs: `["doc_123", "doc_456"]`.
- **Step 2 – Fetch**: Agent calls `fetch_document("doc_123")` using the ID from step 1. Fetch cannot run until search completes.
- **Why Not Concurrent**: If fetch ran before search, it would have no document ID to fetch. The fetch would fail. Sequential is required.

Exam Takeaway: Search-then-fetch is the classic sequential dependency. The exam tests recognizing this pattern across different domains (search then read, list then get).

Concurrent Pattern – Independent Queries

AZURE CLOUD MASTERY

Multiple independent queries can run concurrently when they do not share data and the agent needs all results to form a complete answer.

- **Example – Three Queries**: User asks "Compare weather in Seattle, Boston, and Miami today." Each city's weather is independent of the others.
- **Concurrent Execution**: Agent calls three weather tools simultaneously. Total time = 1 second (longest single call), not 3 seconds.
- **Combining Results**: After all three complete, agent combines results: "Seattle: 65°, Boston: 45°, Miami: 80°." Order of results does not matter.

Exam Takeaway: Concurrent execution reduces latency for multiple independent queries. The exam tests identifying when queries do not share dependencies.

Partial Concurrency – Mixing Both Patterns

AZURE CLOUD MASTERY

Complex workflows can mix sequential and concurrent: run independent groups concurrently, then sequential within each group.

- **Workflow Example**: User asks "Get customer order history and product recommendations, then summarize both." Order history and recommendations are independent (concurrent).
- **Execution Plan**: Concurrently run `get_order_history()` and `get_recommendations()`. Wait for both to complete. Then run `summarize(history, recommendations)` sequentially.
- **Benefit**: Total time = max(order_time, recommendation_time) + summarize_time. Faster than running all three sequentially.

Exam Takeaway: Mix sequential and concurrent to optimize complex workflows. The exam tests designing execution plans with both patterns.

Coding Pattern – Sequential in Python

AZURE CLOUD MASTERY

Sequential execution in Python uses normal function calls – one line after another. The second call only runs after the first returns.

- **Basic Sequential Code**: `result1 = agent1.run(input_data)`, then `result2 = agent2.run(result1)`, then `final = agent3.run(result2)`.
- **Error Propagation**: If `agent1.run()` raises an exception, `agent2.run()` never executes. Handle errors at each step.
- **Time Measurement**: Total time is sum of each step's duration. Use `time.time()` before and after to measure.

Exam Takeaway: Sequential code is straightforward – call functions one after another. The exam tests reading sequential code and identifying the order of execution.

Coding Pattern – Concurrent with asyncio.gather

AZURE CLOUD MASTERY

Concurrent execution in Python uses `asyncio.gather()` to run multiple async functions simultaneously and wait for all to complete.

- **Async Function Definition**: Define agent methods with `async def`. Call them with `await`. Example: `async def get_weather(city): return result`.
- **Gather Syntax**: `results = await asyncio.gather(get_weather("Seattle"), get_weather("Boston"), get_weather("Miami"))`.
- **Result Order**: `results` is a list in the same order as the input functions, regardless of which completed first. `results[0]` is Seattle's weather.

Exam Takeaway: `asyncio.gather` runs async functions concurrently. The exam tests syntax and understanding that results preserve input order.

Concurrent with Timeout – Bounded Waiting

AZURE CLOUD MASTERY

When running concurrent tasks, set a timeout so the agent does not wait forever for a slow or hung tool.

- **Timeout Syntax**: `results = await asyncio.wait_for(asyncio.gather(task1, task2), timeout=5.0)`. Raises `TimeoutError` if not complete in 5 seconds.
- **Partial Results**: If timeout occurs, `gather` cancels all pending tasks. No partial results are returned. Handle with `try/except`.
- **Use Case**: Set timeout = expected longest tool duration + buffer. Example: weather API usually 2 seconds, set timeout 5 seconds.

Exam Takeaway: Timeouts prevent indefinite waiting in concurrent execution. The exam tests setting appropriate timeout values and handling timeout errors.

Concurrent with Exception Handling

AZURE CLOUD MASTERY

When one concurrent task fails, you must decide whether to cancel all tasks or continue with successful ones.

- **Cancel on First Error (default)**: If any task raises an exception, `gather` cancels all other running tasks and re-raises the exception.
- **Return Exceptions Option**: `results = await asyncio.gather(task1, task2, return_exceptions=True)`. Results list contains exception objects instead of raising.
- **Handling Mixed Results**: Loop through results. If `isinstance(result, Exception)`, log error and use default value. Otherwise, use successful result.

Exam Takeaway: `return_exceptions=True` prevents one failure from cancelling all tasks. The exam tests choosing between fail-fast and continue-on-error.

Sequential in Multi-Agent Framework

AZURE CLOUD MASTERY

In Microsoft Agent Framework, sequential execution means calling agents one after another, passing state explicitly between calls.

- **Sequential with Orchestrator**: Create orchestrator. Call `result1 = await orchestrator.run_agent("SearchAgent", query)`. Then `result2 = await orchestrator.run_agent("FetchAgent", result1.document_id)`.
- **State Passing**: The second agent receives the first agent's output as its input. State is not automatic – you must pass it as a parameter.
- **Conversation ID**: Use same `conversation_id` for both calls so agents share conversation history. The orchestrator maintains memory.

Exam Takeaway: Sequential in Agent Framework requires explicit state passing between agent calls using the same conversation ID.

Concurrent in Multi-Agent Framework

AZURE CLOUD MASTERY

In Microsoft Agent Framework, concurrent execution means running multiple agents simultaneously on independent inputs using the same conversation context.

- **Concurrent with Gather**: `results = await asyncio.gather(orchestrator.run_agent("WeatherAgent", "Seattle"), orchestrator.run_agent("WeatherAgent", "Boston"))`.
- **Shared Conversation ID**: Pass same `conversation_id` to both calls. Each agent call sees the same conversation history but works independently.
- **Combining Results**: After all complete, call a third agent sequentially: `await orchestrator.run_agent("SummarizerAgent", results)` to combine outputs.

Exam Takeaway: Use `asyncio.gather` with the orchestrator to run agents concurrently. Use same conversation ID for shared context.

Performance Metrics – Measuring Improvement

AZURE CLOUD MASTERY

Measure execution time before and after implementing concurrency to quantify improvement and identify bottlenecks.

- **Sequential Time Calculation**: Sum of durations of each step. Example: Step A (2s) + Step B (3s) + Step C (1s) = 6 seconds total.
- **Concurrent Time Calculation**: Duration of longest step plus overhead. Example: Max(2s, 3s, 1s) = 3 seconds total. Improvement = 50% faster.
- **Bottleneck Identification**: The slowest concurrent step limits total time. Optimize that step first to improve overall performance.

Exam Takeaway: Concurrent time equals the longest single task duration. The exam tests calculating performance improvement and identifying bottlenecks.

Foundry Trace for Execution Patterns

AZURE CLOUD MASTERY

Foundry Trace shows sequential execution as spans in a chain and concurrent execution as overlapping spans at the same time.

- **Sequential in Trace**: Span A ends, then Span B starts, then Span C starts. No overlap. Duration = sum of all spans.
- **Concurrent in Trace**: Spans start at the same timestamp and overlap in time. Total trace duration = longest span, not sum.
- **Debugging Slow Concurrency**: If concurrent spans are not overlapping (one starts after another ends), your code is accidentally sequential. Check for missing `await` or `gather`.

Exam Takeaway: Read Foundry Trace to verify execution pattern. Overlapping spans confirm concurrency. Sequential chain confirms order dependencies.

Unit 12 Summary – Execution Patterns

AZURE CLOUD MASTERY

You have learned to choose sequential execution for dependent tasks and concurrent execution for independent tasks, then measure performance with Foundry Trace.

- **Sequential**: One step at a time. Use when output of Step A is input to Step B. Total time = sum of durations.
- **Concurrent**: Multiple steps at once. Use when steps are independent. Total time = longest single step duration.
- **Coding Patterns**: Sequential = normal function calls. Concurrent = `asyncio.gather()` with async functions.
- **Observability**: Foundry Trace shows sequential as non-overlapping spans, concurrent as overlapping spans.
- **Next Unit Preview**: Unit 13 introduces Human-in-the-Loop – approval gates and Logic Apps for high-stakes agent actions.

Exam Takeaway: Sequential vs. concurrent is 35% of the exam. Master dependency identification, coding patterns, timeout handling, and trace interpretation.

AZURE CLOUD MASTERY

THANKS FOR WATCHING